

字典 (dict)

1. 为什么需要字典
2. 创建字典
3. 字典的访问、增加和修改
4. 字典的删除操作
5. 字典的遍历
6. 字典的方法
7. 字典的应用

字典 (dict)

为什么需要字典？

- **引入问题**：保存一个学生的信息，包括姓名、年龄、身高、成绩四项
- **尝试**：使用列表

```
student = ['张三', 18, 178, 95]
print(student[1])          # 想取年龄，必须记住它在第 1 个位置
```

必须记住“第 0 个是姓名、第 1 个是年龄、第 2 个是身高、第 3 个是成绩

字典 (dict)

为什么需要字典？

- **引入问题**：保存一个学生的信息，包括姓名、年龄、身高、成绩四项
- **根本原因**：这四项数据不是“一组同类数据”，而是四个带名字的属性，因此需要一种能按“名称”存取数据的类型，这就是字典

名称	值
姓名	张三
年龄	18
身高	178
成绩	95

字典 (dict)

什么是字典？

- 字典：键值对的集合，每个元素由“键”和“值”两部分组成，写作 键: 值。通过键就能找到对应的值，这个过程叫映射

键 (key)	值 (value)
`'name'`	`'张三'`
`'age'`	`18`
`'height'`	`178`
`'score'`	`95`

想取年龄，写 student[‘age’] 即可 (student 为变量名)，不需要记住任何位置

字典 (dict)

什么是字典？

- 字典的特点：

特点	说明
按键访问	不支持索引，通过键取值
键不能重复	一个键只对应一个值
键必须不可变	与集合的元素要求相同
值没有限制	可以是任意类型，包括列表、字典
可变	可以增加、删除、修改键值对

字典 (dict)

什么是字典？

- 键不能重复
  - 一个键只对应一个值，重复的键不会报错，而是后面的值覆盖前面的

```
d2 = {'name': '张三', 'name': '李四'}
print(d2)
```

输出：{'name': '李四'}

字典 (dict)

什么是字典？

- 键必须为不可变类型
  - 字符串、数字、元组可以作为键
  - 列表、集合、字典不能作为键
  - 实际编程中，键绝大多数情况是字符串

```
d1 = {'name': '张三', 18: '年龄', (3, 5): '坐标', 3.14: '圆周率'}
print(d1)
```

输出：{'name': '张三', 18: '年龄', (3, 5): '坐标', 3.14: '圆周率'}

```
d = {['a', 'b']: 1}       # 列表是可变类型，不能作为键
```

输出：TypeError: unhashable type: 'list'

字典 (dict)

什么是字典？

- 值没有任何限制
  - 值可以是任意类型，包括列表和字典

```
student = {
    'name': '张三',
    'scores': [95, 83, 92],                                    # 值可以是列表
    'contact': {'phone': '138xxxx', 'email': 'zs@abc.com'}     # 值可以是字典
}
print(student['scores'][0])                                    # 取第一门成绩
print(student['contact']['phone'])                             # 取电话号码
```

运行结果：

95

138xxxx

字典 (dict)

创建字典：使用大括号

- **语法**：用大括号 {} 把键值对括起来，键和值之间用冒号连接，键值对之间用逗号分隔
- 字典名 = {键1: 值1, 键2: 值2, ..., 键n: 值n}

```
scores = {'张三': 95, '李四': 83, '王五': 92}
print(scores)


empty = {}
print(empty, type(empty))
```

{} 是空**字典**，不是空集合

字典 (dict)

创建字典：其它方式

- **方式一**：使用 dict() 函数

```
student = dict(name='张三', age=18, height=178, score=95)
print(student)
```

- **方式二**：用 zip() 把两个序列配对
  - zip()：把两个序列按位置一一配对：第 1 个名字配第 1 个成绩，第 2 个配第 2 个，以此类推

```
names = ['张三', '李四', '王五']
values = [95, 83, 92]
scores = dict(zip(names, values))
print(scores)
```

⚠ 两个序列长度不同时，以**短的**为准

运行结果：

{'张三': 95, '李四': 83, '王五': 92}

字典 (dict)

创建字典：其它方式

- **方式三**：字典生成式
  - 写法与列表生成式相同，只是方括号换成大括号，并且要写成 键: 值 的形式

```
lengths = {name: len(name) for name in ['Python', 'Java', 'Go']}
print(lengths)
```

运行结果：

{'Python': 6, 'Java': 4, 'Go': 2}

字典 (dict)

■ 字典的访问、修改和增加

- **语法**：字典用中括号加键来存取数据，位置换成了键

```
值 = 字典名[键]                # 取值
字典名[键] = 值                # 赋值
```

```
student = {'name': '张三', 'age': 18, 'height': 178, 'score': 95}

print(student['name'])          # 通过键取值
print(student['score'])

print('age' in student)         # 成员运算：判断键是否存在
print('张三' in student)         # 判断的是键，不是值

student['score'] = 98           # 键已存在：修改对应的值
print(student)

student['class'] = '计算机 1 班'    # 键不存在：增加一个新的键值对
print(student)
```

字典 (dict)

字典的访问、修改和增加

- **语法**：字典用中括号加键来存取数据，位置换成了键

```
值 = 字典名[键]          # 取值
字典名[键] = 值          # 赋值
```

- 字典名[键] = 值 这一个写法同时承担两件事：

情况	效果
键已存在	修改该键对应的值
键不存在	增加一个新的键值对

字典 (dict)

■ 字典的访问、修改和增加

- update() ：一次增加或修改多个键值对

```
student1 = {'name': '张三', 'age': 18, 'score': 95}
student2 = {'age': 19, 'height': 178}

student1.update(student2)     # 合并：键已存在则修改，不存在则增加
print(student1)

student1.update(score=100, city='大连')    # 也可以直接写成键=值的形式
print(student1)
```

运行结果：

{'name': '张三', 'age': 19, 'score': 95, 'height': 178}
{'name': '张三', 'age': 19, 'score': 100, 'height': 178, 'city': '大连'}

字典 (dict)

字典的访问、修改和增加

- 用中括号访问一个不存在的键会报错

```
student = {'name': '张三', 'age': 18, 'score': 95}
print(student['height'])
```

报错 KeyError: 'height'

字典 (dict)

■ 字典的访问、修改和增加

- 用中括号访问一个不存在的键会报错

  ➢ **解决方案一**：使用 get() 方法

```
字典名.get(键)                    # 键不存在时返回 None
字典名.get(键, 默认值)            # 键不存在时返回指定的默认值
```

```
student = {'name': '张三', 'age': 18, 'score': 95}

print(student.get('name'))          # 键存在：返回对应的值
print(student.get('height'))        # 键不存在：返回 None，不报错
print(student.get('height', 0))     # 键不存在：返回指定的默认值 0
```

运行结果：

```
张三
None
0
```

字典 (dict)

字典的访问、修改和增加

- 用中括号访问一个不存在的键会报错
  - **解决方案二**：先用 in 判断

```
if 'height' in student:
    print(student['height'])
else:
    print('没有身高信息')
```

字典 (dict)

字典的删除操作

写法	作用	返回值
`字典.pop(键)`	删除指定的键值对	被删除的**值**
`字典.pop(键, 默认值)`	同上，键不存在时返回默认值，不报错	值或默认值
`字典.popitem()`	删除**最后一个**键值对	被删除的**键值对元组**
`del 字典[键]`	删除指定的键值对	无
`字典.clear()`	清空整个字典	无

字典 (dict)

字典的删除操作

```
student = {'name': '张三', 'age': 18, 'height': 178, 'score': 95}

x = student.pop('height')          # 删除指定的键，并返回对应的值
print(x, student)  178 {'name': '张三', 'age': 18, 'score': 95}

y = student.pop('weight', 0)       # 键不存在时返回默认值，不报错
print(y)  0

item = student.popitem()           # 删除并返回最后一个键值对，结果是元组
print(item, student)  ('score', 95) {'name': '张三', 'age': 18}

del student['age']                 # 用 del 删除指定的键值对
print(student)         {'name': '张三'}

student.clear()                    # 清空字典
print(student)         {}
```

字典 (dict)

字典的删除操作

- 键不存在时的报错：

写法	结果
`student.pop('weight')`	`KeyError: 'weight'`
`student.pop('weight', 0)`	返回 0，不报错
`del student['weight']`	`KeyError: 'weight'`
空字典调用 `popitem()`	`KeyError: 'popitem(): dictionary is empty'`

字典 (dict)

字典的遍历

- 得到全部的键：字典.keys()或直接遍历字典
- 得到所有的值：字典.values()
- 得到所有的键值对，每个键值对是一个元组：字典.items()

```
scores = {'张三': 95, '李四': 83, '王五': 92}

for name in scores:                          # 直接遍历字典，得到的是键
    print(name, scores[name])
print()

for name in scores.keys():                   # 遍历所有的键，与上面等价
    print(name, end=' ')
print()
```

```
张三 95
李四 83
王五 92
张三 李四 王五
```

字典 (dict)

字典的遍历

- 得到全部的键：字典.keys()或直接遍历字典
- 得到所有的值：字典.values()
- 得到所有的键值对，每个键值对是一个元组：字典.items()

```
scores = {'张三': 95, '李四': 83, '王五': 92}
for score in scores.values():                    # 遍历所有的值
    print(score, end=' ')                        95 83 92
print()

for name, score in scores.items():               # 同时遍历键和值
    print(name, score)                           张三 95
                                                 李四 83
                                                 王五 92
```

字典 (dict)

字典的应用

- 字典的典型应用场景：

场景	键	值	实例
按名称查数据	姓名、学号、编号	成绩、电话、价格	成绩单、通讯录、商品价目表
计数统计	被统计的对象	出现次数	词频统计、投票计票、日志分析
保存对象的属性	属性名	属性值	学生信息、商品信息、用户资料
配置参数	参数名	参数值	程序配置、模型超参数
网络数据交换	字段名	字段值	网页接口返回的 JSON 数据
建立对应关系	原始值	转换后的值	国家→首都、英文→中文、编码→名称

字典 (dict)

■ 练习题：学生成绩字典管理

```python
students = {'张三': 85, '李四': 92, '王五': 78}
```

要求：

(1) 用 get() 方法查询"赵六"的成绩（不存在则显示 0）

(2) 修改"李四"的成绩为 96

(3) 删除"王五"

(4) 用 update() 方法新增两个学生："周八" 成绩 88，"孙七" 成绩 91

(5) 遍历输出所有学生的成绩

字典 (dict)

■ 练习题：学生成绩字典管理

```
students = {'张三': 85, '李四': 92, '王五': 78}

# (1)
score = students.get('赵六', 0)
print("赵六的成绩:", score)

# (2)
students['李四'] = 96

# (3)
del students['王五']

# (4)
students.update({'周八': 88, '孙七': 91})

# (5)
for name, score in students.items():
    print(name, score)
```

字典 (dict)

练习题：统计字母出现的次数

- 统计字符串 'Hello World' 中每个字母出现的次数
- 解决思路：

步骤	做法
1	创建空字典，用**字母作键**、**次数作值**
2	遍历字符串中的每个字符
3	是字母才统计，跳过空格等其他字符
4	该字母已统计过：次数加 1；第一次出现：次数记为 1

字典 (dict)

练习题：统计字母出现的次数

- 统计字符串 'Hello World' 中每个字母出现的次数

```
sentence = 'Hello World'
counter = {}                                # 创建空字典保存统计结果
for ch in sentence:
    if ch.isalpha():                        # 判断是否为字母，跳过空格等其他字符
        counter[ch] = counter.get(ch, 0) + 1      # 取出当前次数加 1，没有则从 0 开始
print(counter)

for ch, times in counter.items():           # 遍历输出统计结果
    print(ch, '出现了', times, '次')
```